DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

MariaDB's critical CVEs arrive two months after the patches, and how D1 keeps Postgres off the hot path for millions of databases

Issue #15 | Mon, 19 Oct 2026 | ⏱️ ~45 min read | Level 3/5

🗄️ **Today's two stories**

1. **Between 9 and 17 October, MariaDB published the CVE records for more than a dozen server bugs, three of them rated 9.9 out of 10, all fixed in releases that shipped in August and September.** Any logged-in user, however low their privileges, could trigger them over the network. We explain each of the four worst bugs (a cursor use-after-free, a GIS buffer overflow, a MySQL 5.1-era file-naming escape hatch and a `GRANT PROXY` password takeover), why the CVEs came out weeks after the fixes, and why a public crash proof-of-concept appeared the day after.
2. **Cloudflare explained how D1, its SQLite-based serverless database, copes with millions of small databases, one per user, app or preview environment.** The rule is that no query ever waits on the central Postgres database that holds D1's metadata. We walk through database creation without a transaction, 30-second heartbeats that clean up after failures, a durable routing table on Workers KV, and a pipeline that counts storage without ever running `SUM`.
3. **On the radar:** 8 more things that happened, listed at the end.

STORY 1 MariaDB's late-arriving 9.9s

MARIADB SERVER · CVES PUBLISHED 9–17 OCT 2026 · FIXES SHIPPED 13 AUG–15 SEP · ~20 MIN READ

If you upgraded MariaDB in late August, you may already be protected against three critical vulnerabilities you only heard about this weekend. If you didn't, the instructions for crashing your server went public on Saturday. Between 9 and 17 October, MariaDB published CVE records for a batch of server bugs, three of them scored 9.9 out of 10, that it had quietly fixed in releases two months earlier. Every one of them needs only an ordinary login. And one of them hides in a compatibility feature from the MySQL 5.1 days.

WHAT HAPPENED

On **16 October 2026**, three CVE records for MariaDB Server were published, each scored **9.9 (Critical)**. A fourth, scored **8.8 (High)**, followed on **17 October**.[1,2] All four come from GitHub security advisories on the `MariaDB/server` repository, and all four share the same attacker profile: someone who can **log in**, with low privileges, over the network.[4,5,6,7]

- **CVE-2026-85748 (9.9): a use-after-free in `SYS_REFCURSOR`.** In the advisory's words, "cursor-array reallocation can leave stale references in routine contexts". A low-privileged attacker "can open a large

number of SYS_REFCURSORs and then fetch one to trigger a use-after-free".[4] Affected: **12.3.1–12.3.2 and 13.0.1**. Fixed: **12.3.3 and 13.0.2**.[8]

- **CVE-2026-85746 (9.9): an out-of-bounds write in `ST_AsText`.** The function "can write beyond an allocated output buffer when converting deeply nested GeometryCollections", because the recursive code that writes the text output doesn't track how much space it has used.[5]
- **CVE-2026-92262 (9.9): the `#mysql50#` prefix bypasses file-name encoding.** A table name with this prefix skips MariaDB's normal mapping from table names to file names, so an attacker can reach files "under their legacy unencoded name". The record lists crashes, data corruption, data theft and remote code execution as possible outcomes.[6]
- **CVE-2026-97380 (8.8): `GRANT PROXY` lets you set someone else's password.** The statement "treats an IDENTIFIED clause with an empty password as though authentication were not specified, causing it to check the wrong grantor privileges". A low-privileged attacker "who can name another account can exploit this authorization error to set that account's password".[7]

The last three affect **every supported series**: 10.6.1–10.6.27, 10.11.1–10.11.18, 11.4.1–11.4.12, 11.8.1–11.8.8, 12.3.1–12.3.2 and 13.0.1. The fixed versions are **10.6.28, 10.11.19, 11.4.13, 11.8.9, 12.3.3 and 13.0.2**.[1,7]

They weren't alone. A week earlier, on **9 October**, MariaDB had published another set, CVE-2026-107815 to 107823.[10] These include a one-byte write past a stack buffer in the CONNECT engine's DOS table type (8.5), which could crash the server "and potentially achieve remote code execution", and a set of out-of-bounds reads in the `mysql_json` plugin when it imports MySQL tables with invalid binary JSON (4.4).[12,13] Singapore's Cyber Security Agency (CSA) issued an alert about five of them on 16 October.[14]

What makes this batch unusual is the date of the fixes. MariaDB's Enterprise CVE page lists all four headline CVEs, and 19 others, as fixed in Enterprise Server releases dated **7 September**.[10,11] The community releases came first: **10.6.28 on 13 August**, and **12.3.3, 11.8.9, 11.4.13 and 10.11.19 on 24 August**.[15,17] The 13.0 fix is in **13.0.2**, the release that made 13.0 GA on **15 September**.[20] The patches had been out for seven to nine weeks before the CVE IDs became public.

And then, on **18 October**, a researcher published a proof-of-concept repository for CVE-2026-92262, 85746 and 85748. According to its README, as relayed by Sploitus, it contains "Crash and data-corruption PoCs", tested against a `mariadb:13.0.1-rc` container with a low-privileged user that has no FILE or SUPER rights. It does not include the chain to code execution.[22]

THE BACKGROUND YOU NEED

How a database CVE gets to you

A **CVE** *≈ Common Vulnerabilities and Exposures: a public ID for one specific security flaw, so everyone can refer to the same bug* record is supposed to be the moment the world learns about a bug. Each record has a **CVSS** *≈ Common Vulnerability Scoring System: a 0–10 score built from a "vector" describing how a bug is reached and what it lets an attacker do* score, and the vector behind the score is often more useful than the number. All three 9.9s carry the same vector:[8,5,6]

```
CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H
```

Read left to right: reachable over the **network**, **low** attack complexity, **low** privileges required, no user interaction, **scope changed**, and high impact on confidentiality, integrity and availability. **Scope changed** ≈ *the CVSS judgement that a bug lets the attacker affect things beyond the vulnerable component itself, here the host the database runs on* is what pushes 8.8 to 9.9. The `GRANT PROXY` bug has the same vector with scope unchanged, so it stays at 8.8.[7]

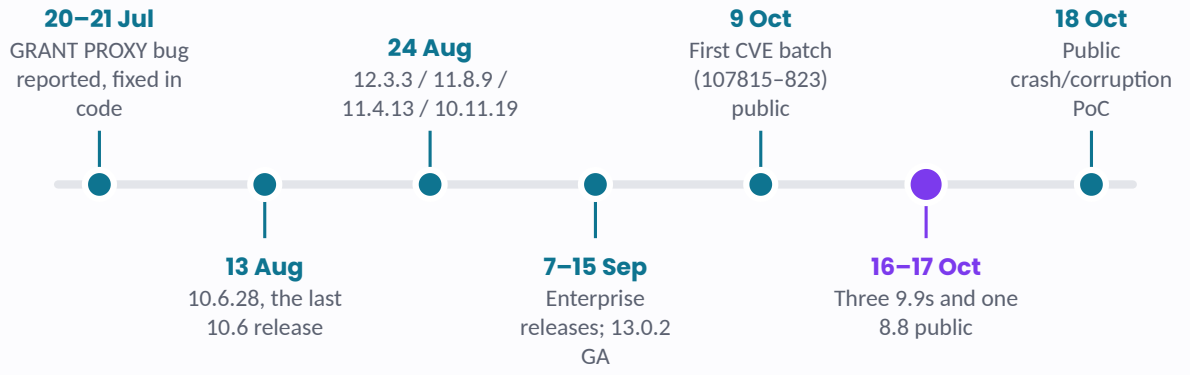
The important word is **low**, not "none". These are not bugs that anyone on the internet can trigger. They need a working account. But "a working account" covers a lot in the MariaDB world: every application user, every reporting login, every tenant on a shared hosting server, and anyone who has found an **SQL injection** ≈ *a bug in an application that lets an attacker smuggle their own SQL into the queries the application sends to the database* in one of your apps. That last case turns a web bug into a "low-privileged database user" for free.

Why the fixes came first

MariaDB doesn't usually put vulnerability details in its release announcements. The Foundation's post on the 24 August releases describes them as bug-fix and stability updates.[17] The release notes carry tables of CVE IDs and CVSS scores, but no descriptions.[18,15] The longer descriptions arrive when the CVE records are published, sometimes weeks later. The Enterprise CVE page says it outright for the 9 October batch: the fixes "were released on 2026-09-07, before their CVE records were published".[10]

You can see the gap in the Debian changelog. When Debian's maintainers uploaded 11.8.9 on **29 August**, they listed six CVEs from an earlier batch, and none of the ones in this story, because those IDs weren't public yet.[21]

From bug report to public crash PoC



Sources: AI Heartland (report and commit dates), MariaDB release notes and Foundation post, MariaDB Enterprise CVE page, CVE records via CVE Brief and TheHackerWire, Sploitus.

This isn't secrecy for its own sake. Shipping the fix before the description gives users a head start before the bug is easy to exploit. The catch is that it only works if people install minor releases they've been told are routine. If you upgraded MariaDB in late August, these CVEs are old news for you. If you only upgrade when you see a CVE, you've had none of the head start, and the PoC is already out.

The release lines you might be on

MariaDB runs **LTS** ≈ *long-term support: a release series that gets bug and security fixes for several years* series alongside short-lived **rolling releases** ≈ *MariaDB's non-LTS series, which get new features and are replaced by the next one rather than patched for years* (we covered the model in Issue #2). As of the Foundation's August post, the maintained community LTS lines are **10.11 (until 2028), 11.4 (2029), 11.8 (2028) and 12.3 (2029)**.[17]

10.6 is over. It reached community end of life on **6 July 2026**, and 10.6.28 "is the final release of the MariaDB 10.6 series".[17,16] If you're on 10.6.27 or earlier, 10.6.28 fixes this batch, and there won't be a 10.6.29 for the next one. Plan the move to 10.11 or later now.

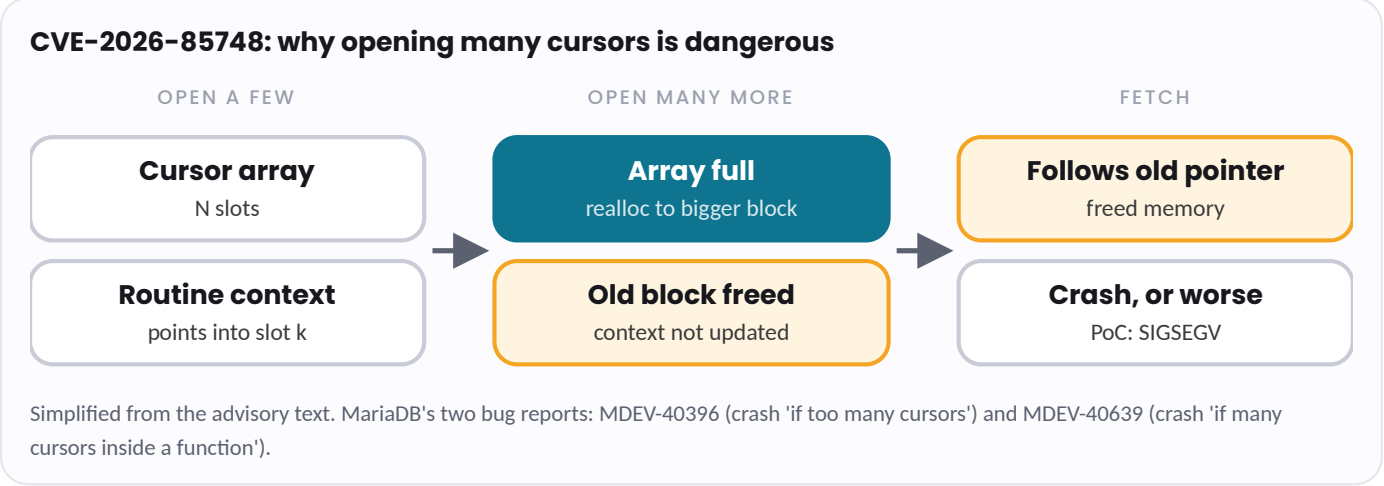
HOW IT WORKS

Here's what each of the four headline bugs actually does. The advisories are short, so where we go beyond them we say so.

1. `SYS_REFCURSOR`: a pointer that outlives its array (CVE-2026-85748)

A **cursor** ≈ *a handle that lets a stored program walk through a query's result one row at a time* in a stored procedure normally belongs to one fixed query. `SYS_REFCURSOR` is the Oracle-style alternative: a "weak" **ref cursor** ≈ *a cursor variable that can be opened for any query at run time and passed between procedures, as in Oracle PL/SQL*. It was added to MariaDB under MDEV-20034, "Add support for the pre-defined weak SYS_REFCURSOR", and a bug report filed against 12.0 shows it arrived in the 12.x series.[23] The CVE only affects 12.3.1 onwards and 13.0.1, which tells you how new the code is.[8]

The bug is a classic of C and C++ programs. The server keeps the open ref cursors of a session in an array. When you open more cursors than the array has room for, the server **reallocates** it: it allocates a bigger block of memory, copies the entries over and frees the old block. Any code that still holds a pointer into the old block now points at freed memory. That's a **use-after-free** ≈ *a memory-safety bug where a program keeps using memory after releasing it, so it reads or writes whatever now lives there*. Per the advisory, the stale pointers sit in "routine contexts", the per-call state of stored routines, and fetching from a cursor follows them.[4]



Simplified from the advisory text. MariaDB's two bug reports: MDEV-40396 (crash 'if too many cursors') and MDEV-40639 (crash 'if many cursors inside a function').

The two Jira tickets in the 12.3.3 changelog say the same thing in bug-report language: "open SYS_REFCURSOR crash, if too many cursors" and "Open SYS_REFCURSOR crash, if many cursors inside a function".[19] The public

PoC's README reports a segmentation fault in `Materialized_cursor::open`.[22]

2. `ST_AsText`: recursion with no budget (CVE-2026-85746)

MariaDB's **GIS** ≈ *geographic information system features: spatial column types like POINT and POLYGON, plus functions to build and query them* functions include `ST_AsText`, which turns a stored geometry into **WKT** ≈ *Well-Known Text: the standard text form of a geometry, like POINT(1 2) or GEOMETRYCOLLECTION(...)*. A **GeometryCollection** ≈ *a geometry that contains other geometries, including other collections, so it can be nested many levels deep* can contain other collections, so the code that writes the text calls itself for each nested level.

According to the CVE, that recursive code doesn't keep track of how many bytes it has written or reserve enough room in the output buffer, so a deeply nested collection makes it write past the end.[5] That's an **out-of-bounds write** ≈ *writing past the end of a buffer into memory that belongs to something else*, the most dangerous kind of memory bug, because the attacker chooses what gets written. The PoC README reports `free(): invalid next size` and an abort, which is the memory allocator noticing that its own bookkeeping has been overwritten.[22]

If you run MariaDB as an application database, you may think you don't use GIS. That doesn't protect you: `ST_AsText` is a built-in function, and the attacker in the advisory is any low-privileged user who can supply a crafted geometry, not someone who needs GIS columns in your schema.[5]

3. `#mysql50#`: an escape hatch from MySQL 5.1 (CVE-2026-92262)

This one needs some history. MariaDB stores each database as a directory, and many storage engines store each table as one or more files named after it. File systems don't allow every character, so table names go through an encoding. Letters, digits and underscores are kept as they are, and everything else becomes an `@`-sequence. The docs' example: the table `this_is_таблица` becomes the file `this_is_@y0@g0@h0@r0@o0@i1@g0`.[24] In MySQL's own docs, a hyphen becomes `@002d`, so a database called `a-b-c` lives in the directory `a@002db@002dc`.[26]

This encoding arrived in **MySQL 5.1**. Before that, names were written to disk as they were. To let people open tables created by older versions, the server shows such legacy names with a special prefix: "the server prefixes mysql50 to their name", and "the prefix needs to be supplied to reference this table".[25] An old table named `table@1` shows up as `#mysql50#table@1`. The upgrade tools used it too: `ALTER DATABASE` #mysql50#a-b-c `UPGRADE DATA DIRECTORY NAME` re-encodes an old directory name.[26]

In other words, `#mysql50#` means "use this name on disk exactly as written, with no encoding". That is a **path traversal** ≈ *a bug class where an attacker-chosen name is used to reach files outside the place they were meant to stay*-shaped feature, and CVE-2026-92262 is what happens when it isn't fenced in. The advisory says the prefix "bypasses normal table-name filesystem encoding", letting an attacker reach files under their unencoded name.[6] The record classifies it as CWE-706, "use of incorrectly-resolved name or reference".[3]

The simplest consequence is a name collision. Here is an illustration we've derived from the documented rules, not the PoC's exact payload:

Two SQL names, one file (illustrative)

Normal encoding

```
CREATE TABLE `a-b` (...);
-- name contains '-'
-- encoded on disk as:
--   a@002db.ibd
```

Legacy prefix, no encoding

```
CREATE TABLE `#mysql50#a@002db` (...);
-- name used on disk as written:
--   a@002db.ibd
-- same file as `a-b`
```

Derived from MariaDB/MySQL identifier-mapping docs. The public PoC reports 'rows inserted under one name appear in another table'.

That's exactly what the PoC README reports: a `#mysql50#` name collision where "rows inserted under one name appear in another table", silently and without a crash.[22] An attacker who can create a table in their own database can then, in principle, read or write a table they have no rights to. And names that never get encoded can include characters the encoding was designed to keep out of file names. The advisory's list of outcomes, which runs as far as remote code execution, reflects that.[6]

4. `GRANT PROXY`: checking one node, writing all of them (CVE-2026-97380)

Proxy users *≈ a MariaDB feature that lets one account log in and act with another account's privileges, often used with single sign-on plugins* are set up with `GRANT PROXY ON target TO grantee`. As with `CREATE USER`, the grantee can carry an `IDENTIFIED` clause, which sets a password or authentication method, and MariaDB lets you chain several authentication methods in one clause.

This is the bug with the best public write-up. It was reported through MariaDB's **bug bounty** *≈ a program that pays outside researchers for responsibly reported vulnerabilities; MariaDB runs one on HackerOne* program on HackerOne. ITmedia's @IT attributes the report to Assetnote and links to a post by researcher Kevin Gervot ("Mizu"). AI Heartland names the reporter as kevin_mizu, says it was reported on 20 July, fixed in code on 21 July, and that the HackerOne report was disclosed on 7 September.[27,28] According to both:

- The privilege check (`copy_and_check_auth`, using `LEX_USER::has_auth`) looked only at the **first** authentication node. If that node was empty, the server decided no authentication was being set and skipped the check that you're allowed to change that account.[27,28]
- The code that saves the result, `replace_user_table()`, wrote **every** node. So a later node's password was stored on the target account.[28]
- Because MariaDB lets you grant PROXY on your own identity, a user with only `USAGE ON *.*`, the bare minimum, could reach this code. They needed no `ALTER USER`, `CREATE USER` or `mysql` schema access. The target account had to exist and match the attacker's connection host.[27,28]

CVE-2026-97380: the check and the write disagree



From the @IT and AI Heartland write-ups of the HackerOne report. Neither publishes the exploit statement, and neither do we.

The attacker then logs in as, for example, the DBA account, which keeps all its privileges.[27] The fix (commit `dbd60d0ad8da`) makes `has_auth()` check whether an authentication node *exists* rather than whether it has content, and stops the parser creating an empty authentication object for `current_user`.[28] It's the same family of bug as PgBouncer's "second startup message" in Issue #14: two parts of the code with different ideas of what the request says.

The four headline CVEs, by CVSS 3.1 base score



Scores from the CVE records (via TheHackerWire, CVE Brief) and MariaDB's Enterprise CVE page, viewed 19 Oct 2026.

WHAT IT MEANS FOR YOU

Find out what you run. `SELECT VERSION();` takes five seconds. You're fixed if you're on **10.6.28, 10.11.19, 11.4.13, 11.8.9, 12.3.3, 13.0.2** or later on your series.[7] Anything older on those lines is affected by at least three of the four.

Then check your packages, not just upstream. Upstream fixed this in August; your distro may not have caught up.

- **Debian:** when we checked the tracker for CVE-2026-107815 on 19 October, **trixie (stable) was at 11.8.6-0+deb13u1, marked vulnerable**, while forky and sid had 11.8.9. The same page listed bookworm's security build, `10.11.19-0+deb12u1`, as vulnerable even though 10.11.19 is the upstream fixed version, so the tracker seems to be lagging. Check the installed package version yourself.[12]
- **Amazon RDS for MariaDB** has offered 10.6.28, 10.11.19, 11.4.13, 11.8.9 and 12.3.3 since **8 September**, saying they include fixes for "Common Vulnerabilities and Exposures (CVEs) in prior versions of MariaDB". AWS recommends upgrading, using Blue/Green Deployments, an in-place upgrade or automatic minor version upgrades.[29]

- **Containers:** check the tag. The PoC was tested against `mariadb:13.0.1-rc`, the kind of image that sits untouched in a `docker-compose.yml` for months.[22]

Decide how urgent it is for you. All four bugs need a login, so the real question is who has one.

- **High urgency:** shared hosting, platforms that give each customer a database account, internal databases with many human logins, and any database behind an app that might have an SQL injection bug. In those cases "low privileges" is already in the attacker's hands.
- **Lower but real:** a database with one application account and no direct access. An attacker still needs that account, but if they get it, these bugs can turn data access into a crashed server or worse.

If you can't upgrade this week, reduce what an ordinary account can do. The public SYS_REFCURSOR PoC works through stored procedures, so accounts that can't create routines are harder to use.[22] Accounts that can't create tables can't plant a `#mysql50#` name. These are mitigations, not fixes: the `GRANT PROXY` bug needs nothing beyond `USAGE`.[27] The only complete fix is the new version.

> 💥 **Watch out for**
>
> - **The release notes and the CVE records don't fully agree.** The 10.6.28 notes list CVE-2026-92262 but not 85746 or 97380, and the 11.8.9 notes list 92262 and 97380 but not 85746, yet the CVE records name all of those versions as fixed.[15,18,5,7] The 13.0.2 notes list no CVEs at all.[20] Go by the CVE records' fixed-version lists, and if your compliance process reads release notes, expect questions.
> - **One CVE description is confusingly worded.** CVE-2026-85748 says "From 12.3.1 until 12.3.3 and 13.0.2"; the structured version table says ≥12.3.1 <12.3.3 and ≥13.0.1 <13.0.2.[9,8] 12.3.3 and 13.0.2 are fixed.
> - **Some third-party pages overstate it.** One vulnerability site's FAQ says CVE-2026-85748 needs no authentication, which contradicts its own vector (`PR:L`).[4] Every record in this story needs a login.
> - **"No public exploit" is out of date.** Several CVE pages said so on 16–17 October. The crash and corruption PoC appeared on 18 October.[22]
> - **The PoC isn't a code-execution exploit,** and the remote code execution in the descriptions is a "potential" outcome. Don't overreact, but don't wait for a weaponized version either.

WHAT HAPPENS NEXT

The Enterprise CVE page still lists several IDs fixed in the same releases as "Pending", including CVE-2026-85745, 85985, 86047 (7.7) and 102523 (8.0).[10] Expect more records in the coming days, with descriptions of bugs that are already fixed. Distributions will catch up, starting with Debian stable. And 10.6 users should note that the next batch will find them without a patch.

The bigger pattern is the one we've followed since Issue #13. Old C and C++ code that parses untrusted input (a cursor array, a geometry, a file name, a list of authentication options) is being audited faster than ever, by bounty hunters and increasingly by AI-assisted tools. The response that works is boring: install minor releases when they ship, not when the CVE arrives.

📖 Words from this story

CVE: Common Vulnerabilities and Exposures, a public ID for one specific security flaw.

CVSS: Common Vulnerability Scoring System, a 0–10 score built from a vector describing how a bug is reached and what it allows.

Scope changed: the CVSS judgement that a bug lets an attacker affect things beyond the vulnerable component, such as the host.

SQL injection: an application bug that lets an attacker smuggle their own SQL into the application's queries.

LTS: long-term support, a release series that gets fixes for several years.

Rolling releases: MariaDB's non-LTS series, which are replaced by the next one rather than patched for years.

Cursor: a handle that lets a stored program walk through a query result one row at a time.

Ref cursor: a cursor variable that can be opened for any query at run time and passed around, as in Oracle PL/SQL.

Use-after-free: a memory-safety bug where a program keeps using memory after releasing it.

GIS: geographic information system features, meaning spatial types and functions.

WKT: Well-Known Text, the standard text form of a geometry.

GeometryCollection: a geometry that contains other geometries, possibly nested many levels deep.

Out-of-bounds write: writing past the end of a buffer into memory that belongs to something else.

Path traversal: a bug class where an attacker-chosen name reaches files outside the place it was meant to stay.

Proxy users: a MariaDB feature that lets one account act with another account's privileges.

Bug bounty: a program that pays outside researchers for responsibly reported vulnerabilities.

STORY 2 D1 and the million tiny databases

CLOUDFLARE D1 · ENGINEERING POST, 13 OCT 2026 · ARCHITECTURE CASE STUDY · ~20 MIN READ

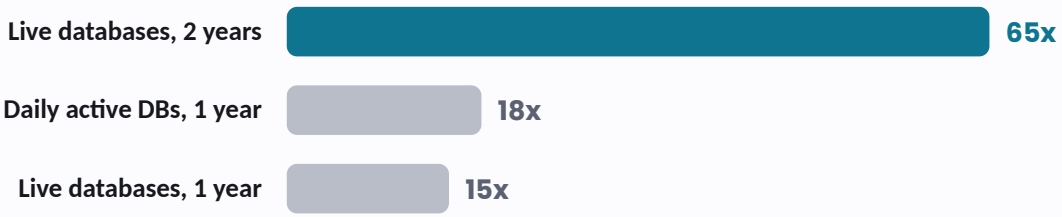
For thirty years, the database was the big, expensive thing in the middle of the diagram. You had one, maybe two, and you guarded them. Then AI app builders started giving every user, every generated app and every preview branch its own database. Cloudflare's D1 team now runs millions of them, hundreds of thousands active at once, and last week it explained the design rule that makes this work: the central database that knows about all the other databases must never be on the path of a single query.

WHAT HAPPENED

On **13 October 2026**, Cloudflare engineers David Laban and Lambros Petrou published "Under the hood, off the hot path: How D1 scales to millions of databases", a 24-minute deep dive into D1's control plane.[30] D1 is Cloudflare's **serverless** *≈ a service you use without provisioning or managing servers, billed by usage* SQL database built on **SQLite** *≈ a small, embedded SQL database engine that stores a whole database in one file and runs inside the application process*. It became generally available in April 2024.[31,32]

The post starts with a change in how databases are used. "Traditional database architecture assumes you have one or two big database clusters per application," it says, but AI-assisted development and vibe-coding platforms are "driving applications toward many small databases: one per user, application, preview environment, or tenant".[30] The growth figures, as of **8 October 2026**, are relative, with no absolute counts:[30]

D1 growth multiples, as of 8 Oct 2026



Cloudflare's own figures (vendor claims). 'Active' = ran at least one query that day. The post gives no absolute counts beyond 'millions of created databases, hundreds of thousands of which are active at any point'.

The post quotes Jon Abrams of ChatGPT Sites at OpenAI, which uses D1 for custom metrics, leaderboards and comments on generated sites.[30] That's the use case in one line: an AI builds an app, and the app needs somewhere to keep its state.

THE BACKGROUND YOU NEED

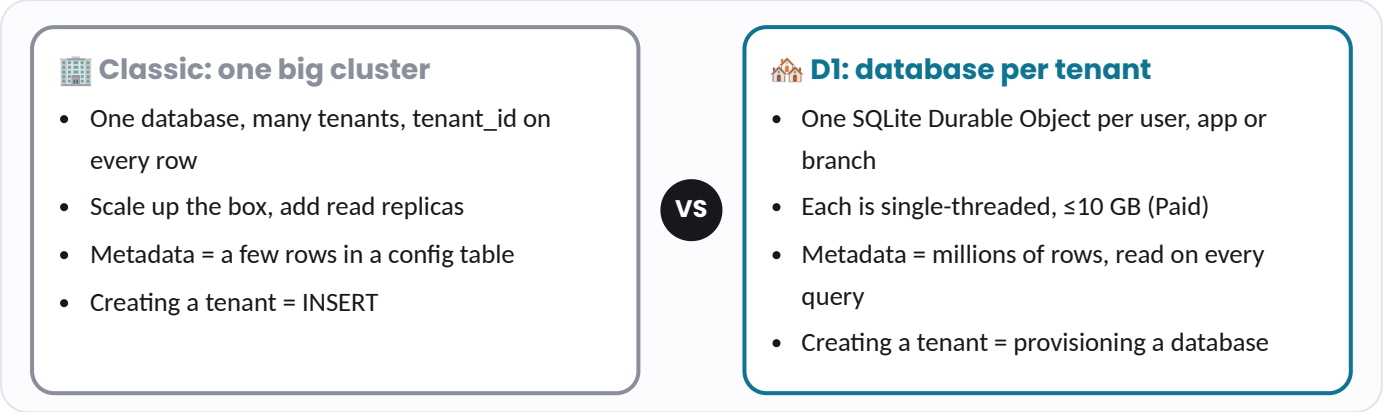
One database, one Durable Object

D1's trick for having millions of databases is to make each one tiny and self-contained. "Each D1 database is backed by one SQLite Durable Object, which makes it very easy to create millions of them."[30]

A **Durable Object** ≈ *a Cloudflare primitive: a single-threaded program instance with a globally unique ID and its own private, persistent storage, running in one data center at a time* (DO) is a small stateful program that runs on Cloudflare's network. Since 2024 each one can have its own SQLite database. SQLite runs as a library inside the object, so "your storage lives in the same thread as the application, requiring not even a context switch to access".[33]

Durability doesn't come from one disk. A Storage Relay Service captures SQLite's **write-ahead log** ≈ *WAL: a file where a database first records each change before applying it, so changes can be replayed after a crash* and, on every commit, forwards changes to five follower machines in different data centers. A write is confirmed once "at least three of these followers respond". That's a **quorum** ≈ *the minimum number of replicas that must acknowledge a write before it counts, here 3 of 5*.[33] An **output gate** ≈ *a Durable Objects mechanism that holds a response back until the writes it depends on are confirmed durable* holds the response until that has happened.[33] The log also goes to object storage in batches, which is what gives D1 point-in-time recovery.[33]

The result: each D1 database is a single-threaded SQLite instance in one location, with replicated storage under it. The docs are clear about what that means for throughput. Each database "handles one query at a time", so a 1 ms average query allows about 1,000 queries per second and a 100 ms query about 10.[34] This is not a design for one huge database. It's a design for many small ones.



The problem: metadata for millions

Having millions of databases moves the hard part. Every query has to answer "which Durable Object is this database, and is this user allowed to touch it?" Every write has to answer "is this account over its storage limit?" Someone has to know which databases exist, who owns them and how big they are. That someone is D1's **control plane** ≈ *the part of a system that manages resources (create, delete, configure, enforce limits) rather than serving user requests*: Postgres in Cloudflare's central "Core" data centers, which holds the transactional metadata.[30]

Postgres is good at that. What it can't be is on the path of every query, from every continent, for millions of databases. That would make one region's database a dependency of every D1 query in the world. The post's guiding principles say so directly: "Avoid centralized single-region components in any data-plane hot path, including all query requests."[30]

So D1 splits its work into three **planes** ≈ *groups of operations with different speed and consistency needs, each allowed different dependencies*:[30]

- **Data plane:** "any database query related request; should be as fast as possible and always at the edge."

- **Control plane:** "any HTTP API operation or internal flows that run in response to a non-query user request". These may depend on central components.
- **Background asynchronous plane:** "any analytical and billing-related workflow", which can be eventually consistent and run anywhere.

HOW IT WORKS

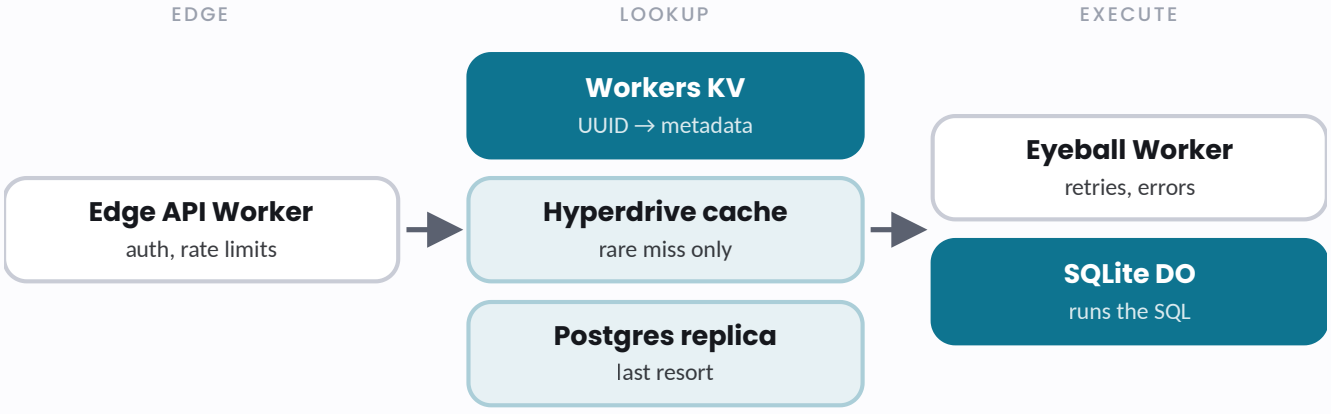
The query path never touches Postgres

D1's request handling is built from Cloudflare **Workers** *≈ Cloudflare's serverless functions, which run in every data center close to the user*. The **Edge API Worker** handles HTTP API traffic. The **Eyeball Worker** is a proxy in front of every Durable Object, which centralizes retries, error handling and observability.[30]

The routing table, which maps a database's UUID to its metadata (including its Durable Object ID), lives in **Workers KV** *≈ Cloudflare's globally replicated key-value store, optimized for many reads and few writes*. A query over HTTP goes like this:[30]

1. The Edge API reads the UUID-to-metadata mapping from Workers KV.
2. On a rare miss, it falls back to **Hyperdrive** *≈ Cloudflare's connection pooler and query cache for databases, run close to both the caller and the database*, reading Postgres **read replicas**. Hyperdrive checks its cache near the caller, then near Postgres, and only then queries a replica.
3. The Edge API calls the Eyeball Worker through a **service binding** *≈ a direct call from one Cloudflare Worker to another, without going over the public internet*.
4. Eyeball validates the request and forwards it to the database's Durable Object.
5. The DO runs the SQL and the result goes back the same way.

A D1 query over HTTP: the hot path



From Cloudflare's D1 control-plane post. The Postgres primary never appears on this path.

Queries from a Worker skip even the lookup. A D1 **binding** *≈ configuration that gives a Worker a ready-made handle to a resource such as a database* is a service binding to Eyeball that carries encoded authorization for one specific database, so binding queries "skip the UUID-to-metadata lookup and the Edge API auth checks".[30]

Why KV and not just a cache in front of Postgres? Because of what happens on a bad day. The post calls Workers KV "a durable datastore, not just a cache". If routing data were only cached, a cache purge would

mean "a million databases trying to fetch their routing information", which "would lead to metastable failures".[30] A **metastable failure** *≈ a failure where a trigger pushes a system into a bad state that persists even after the trigger is gone, often because recovery itself creates overload* is the kind where the system stays down after the original cause is gone. Marc Brooker sums up the research definition: it is "a bad state that persists even when the trigger is removed".[37] A cold cache that floods the database, which then can't refill the cache, is a textbook example.

The post makes one more choice explicit. D1 gives each database's DO a random ID with `newUniqueId()` and stores that ID in its metadata, rather than deriving the DO from the database's name. Name-based addressing would mean two lookups instead of one. The authors say named DOs are still the right starting point for most applications.[30]

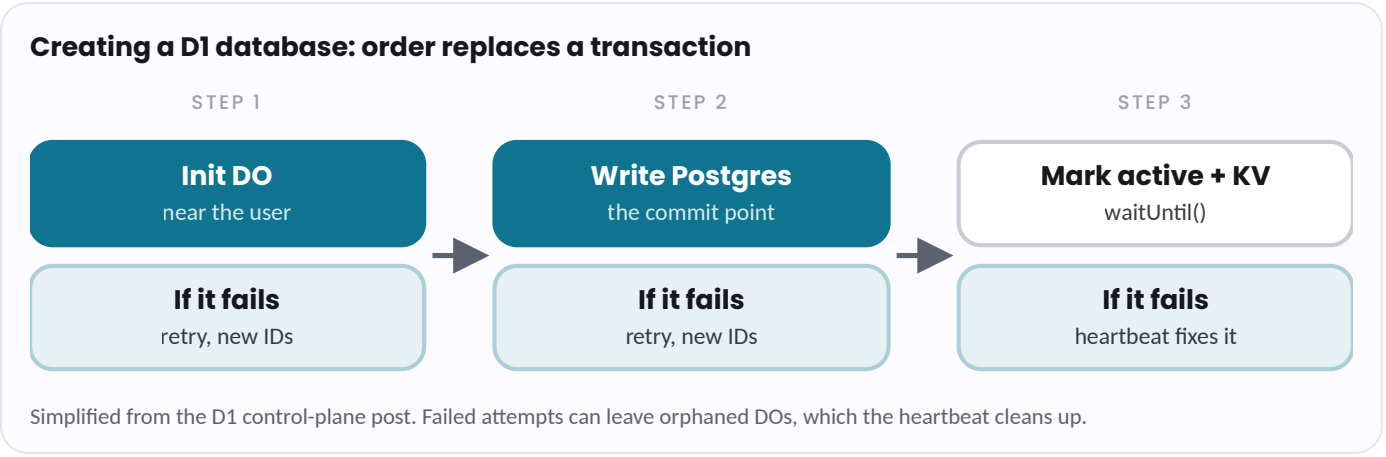
Creating a database without a transaction

Creating a database touches three systems: a new Durable Object somewhere in the world, a metadata row in central Postgres and a routing entry in Workers KV. The obvious approach is to wrap it in one Postgres **transaction** *≈ a group of database changes that either all happen or none do*. D1 doesn't, because "it leads to connections being held while the Durable Object is being initialized, which could be on the other side of the world".[30]

Instead, creation is a sequence of steps, each safe to retry:[30]

1. Check account limits. Generate a database UUID and a fresh DO ID.
2. **Initialize the DO first**, near the requester by default, or wherever a location hint or **jurisdiction** *≈ a constraint that keeps data within a legal region, such as the EU* says.
3. **Then write the metadata row to Postgres.** This is the commit point: "a database is considered created even if the final response fails to reach the user."
4. In the background (`waitUntil()`), tell the DO it's active and write the KV mapping. This step is allowed to fail.

The order matters. Because the DO exists before Postgres points at it, a query can never be routed to a database that isn't there. If step 2 or 3 fails, the flow retries with **new** IDs, never the old UUID with a new DO, because that "would let multiple DOs claim the same database".[30] This is **idempotency** *≈ the property that repeating an operation has the same effect as doing it once, so retries are safe* by design rather than by locking. The post's first principle is to design every process "for failure and idempotency".[30]



Simplified from the D1 control-plane post. Failed attempts can leave orphaned DOs, which the heartbeat cleans up.

Heartbeats: letting the system repair itself

Retrying with new IDs leaves debris: a DO that was initialized but never recorded. D1 cleans it up with a **heartbeat** ≈ *a periodic "I'm alive, what's my status?" message a component sends so the rest of the system can detect and correct problems*. Each DO checks in with the control plane, and "the interval is 30 seconds with a per-database jitter on top".[30] The **jitter** ≈ *a small random delay added to periodic work so that millions of instances don't all fire at the same moment* keeps millions of databases from all checking in on the same second.

The reply tells the DO whether it's active or deleted, and its account's limits. The post's example: DO A is initialized, but the Postgres write fails. The retry creates DO B, and Postgres records B. "Within 30 seconds, DO A sends its next heartbeat", is told the database doesn't exist, and marks itself deleted.[30] Each DO also keeps its own active or deleted flag, so it can accept or reject queries immediately even if Workers KV, which is **eventually consistent** ≈ *a guarantee that all copies will agree once updates stop, but reads may briefly see old data*, is stale.[30]

Because heartbeats can tolerate slightly old data, they're served from Hyperdrive caches and Postgres read replicas, not the primary.[30] So the bulk of the heartbeat traffic never reaches the primary.

Counting storage without `SUM`

The free plan allows 5 GB of D1 storage per account and Workers Paid allows 1 TB, which can be raised on request.[30,34] Enforcing that needs the total for each account. The naive query is a `SUM` over all of an account's databases, and a platform account can have hundreds of thousands of them. A query that gets slower with every new customer database, run on every heartbeat, would sink the control plane.

D1 does the arithmetic once, in the background:[30]

1. Each DO publishes size updates to a queue, aggregated per edge server first. The number of queue consumers caps the write load on Postgres.
2. A **trigger** ≈ *database code that runs automatically when a row is inserted, updated or deleted* on `database_stats` computes each database's size change and inserts the **delta** into `account_database_stats_changes`.
3. A scheduled job **every second** consumes the deltas with a **CTE** ≈ *common table expression: a named subquery introduced with WITH, which in Postgres can also delete or insert rows* using `DELETE ... RETURNING`, sums them per

account and **upserts** ≈ *inserts a row, or updates it if one with the same key already exists (INSERT … ON CONFLICT DO UPDATE in Postgres)* the result into `account_database_stats`.

Account storage: scan vs precomputed (Postgres)

Naive: on every check

```
SELECT SUM(size)
FROM database_stats
WHERE account_id = $1;
-- cost grows with every
-- database the account owns
```

D1-style: once a second

```
WITH moved AS (
  DELETE FROM account_database_stats_changes
  RETURNING account_id, delta)
INSERT INTO account_database_stats
  (account_id, storage)
SELECT account_id, SUM(delta)
FROM moved GROUP BY account_id
ON CONFLICT (account_id) DO UPDATE
SET storage = account_database_stats.storage
            + EXCLUDED.storage;
```

Right pane reconstructed from fragments quoted in the post; the column name delta and the exact shape are ours.

Summing deltas into one row per account in a single statement also avoids row-lock contention on large accounts. The post is careful to say this is eventually consistent, "not true incremental view maintenance".[30] The heartbeat then reads one row:

```
-- $2 = Math.floor(Date.now() / (X * 1000))
SELECT storage, $2 as dynamic_ttl
FROM account_database_stats WHERE account_id = $1
```

That's a small trick. Hyperdrive caches read-only query results for 60 seconds by default (`max_age`), with up to 1 hour allowed.[35] The cache key includes the parameters, so adding a value that changes every X seconds gives D1 a cache that refreshes every X seconds without any cache configuration.[30] Because the key uses only the account ID, every database in an account shares one cache entry. Precision is traded for hit rate: a database that pushes its account over the limit is blocked a little later than it would be with a per-database key, which the team accepts.[30]

History goes to **ClickHouse** ≈ *a column-oriented database built for fast analytical queries over large volumes of events*. Periodic snapshots of Postgres storage go there, and ClickHouse events are the billing source of truth.[30]

Row limits enforced at the edge

Free-tier daily read and write limits use a different pattern that skips Postgres completely.[30] Each DO counts rows read and written per query, index updates included, and sends a usage event to ClickHouse. A central "Limiter" queries ClickHouse for accounts over their daily limit. For a free account that's over, it emails the owner and sets a key in **Quicksilver** ≈ *Cloudflare's internal globally replicated key-value store, used for configuration that*

must reach every data center fast that expires at the end of the day. Every DO checks Quicksilver locally and rejects queries while the key is set. Upgrading to Workers Paid clears it.

Quicksilver is fast enough for this. Cloudflare's numbers for Workers KV Instant, a new mode built on it that launched in private beta on 1 October, are a median of 107 ms and a p99 of 256 ms to replicate a write to all edge locations, and a p99 read of 1.62 ms. Those are vendor figures.[36] The post suggests developers can use the same pattern with KV Instant.[30]

WHAT IT MEANS FOR YOU

Most of you aren't running a million databases. But "database per tenant" has stopped being exotic, because AI app builders, preview environments and per-customer isolation all push toward it. And D1's lessons apply to any system with a central metadata store and a busy data path.

1. **Keep the central database off the hot path.** Ask which systems every request has to touch. If the answer includes a single-region primary, a regional outage takes everything down with it. D1 puts routing in a durable, replicated store (KV) and treats Postgres as the source of truth that is rarely read on the request path.
2. **Don't let a cache be your only copy of critical data.** A cache that can be purged is a potential thundering herd. If losing the cache would flood the origin, the data belongs in a store that survives the purge.
3. **Replace distributed transactions with ordering and reconciliation.** Create the dependent resource first, make one write the commit point, and run a periodic process that cleans up whatever failure leaves behind. That's how D1 avoids holding a Postgres transaction open across the planet.
4. **Precompute aggregates you check often.** A trigger that writes deltas and a job that folds them in once a second is plain Postgres you can copy. Accept that the total can be a second or two behind.
5. **Turn cache freshness into a cache key.** The `dynamic_ttl` trick works with any cache that keys on query parameters.

If you're considering D1 itself, read the limits page first. On Workers Paid: 50,000 databases per account (raisable), 10 GB per database, 1 TB per account (raisable), 30 days of Time Travel. On Free: 10 databases, 500 MB each, 5 GB total.[34] And remember each database is single-threaded: the per-database ceiling is set by query duration, and when the queue overflows you get an "overloaded" error.[34]

💥 Watch out for

- **Vendor post, relative numbers.** No absolute counts, no hot-path latencies, no costs.[30]
- **Limits lag.** Totals pass through a queue, a trigger, a per-second job and an account-wide cache, so an account can briefly overshoot, by design.[30]
- **KV is eventually consistent,** which is why each DO keeps its own active/deleted flag. Don't treat KV reads as exact.[30]
- **One hot tenant still hits the single-threaded ceiling.**[34]
- **Query execution and read replication (beta since April 2025) aren't covered;** the post points to a separate write-up.[30,31]

WHAT HAPPENS NEXT

The post says Cloudflare uses the edge-enforcement pattern (usage events into ClickHouse, a central limiter, a Quicksilver key checked at the edge) across the company, and that other D1 limits may move to it.[30] The broader question is whether "a database per thing" becomes a default, the way "a container per service" did. Supabase agreeing to buy Turso, the SQLite-based company, earlier this month (on our radar in Issue #9) suggests D1 won't be the only one building for it.

📖 Words from this story

Serverless: a service you use without managing servers, billed by usage.

SQLite: a small embedded SQL engine that keeps a whole database in one file inside the application process.

Durable Object: a single-threaded Cloudflare program instance with a unique ID and its own persistent storage.

Write-ahead log (WAL): a log where each change is recorded before it's applied, so it can be replayed after a crash.

Quorum: the minimum number of replicas that must acknowledge a write, here 3 of 5.

Output gate: a mechanism that holds a response until the writes it depends on are durable.

Control plane: the part of a system that manages resources rather than serving user requests.

Planes: groups of operations with different speed and consistency needs.

Workers: Cloudflare's serverless functions, running in every data center.

Workers KV: Cloudflare's globally replicated key-value store for read-heavy data.

Hyperdrive: Cloudflare's connection pooler and query cache for databases.

Service binding: a direct call from one Worker to another.

Binding: configuration that gives a Worker a handle to a resource such as a database.

Metastable failure: a bad state that persists after its trigger is gone, often because recovery creates overload.

Transaction: a group of database changes that either all happen or none do.

Jurisdiction: a constraint that keeps data within a legal region.

Idempotency: repeating an operation has the same effect as doing it once.

Heartbeat: a periodic status check-in used to detect and correct problems.

Jitter: a small random delay that keeps periodic work from firing all at once.

Eventually consistent: copies agree once updates stop, but reads may briefly be stale.

Trigger: database code that runs automatically on insert, update or delete.

CTE: a named subquery introduced with WITH; in Postgres it can also modify rows.

Upsert: insert a row, or update it if the key already exists.

ClickHouse: a column-oriented database for fast analytics over large event volumes.

Quicksilver: Cloudflare's internal globally replicated key-value store for fast-propagating configuration.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **DynamoDB Streams `AT_TIMESTAMP` (15 Oct):** consumers can start reading a stream at a chosen time within the 24-hour retention window, for replay, recovery or skipping a backlog, at no extra charge beyond GetRecords.[38]
- **Snowflake Iceberg v3 upgrades GA (16 Oct):** `ALTER ICEBERG TABLE ... UPGRADE TO ICEBERG VERSION 3` upgrades tables in place, and Snowflake warns that "Upgrading to v3 is permanent".[39]
- **Cloudflare Hyperdrive MySQL caching bug (14 Oct):** client and endpoint nodes cached MySQL results in different encodings, which produced empty results about 0.001% of the time; Hyperdrive now always caches one canonical encoding.[40]
- **pg_hint_plan 1.9.0 (18 Oct):** PostgreSQL 19 support and path generation strategies; 1.4.5 is the last planned release for PG 14.[41]
- **Confluent Tableflow (16 Oct):** now bills for index-related processing when tables use the upsert write mode.[42]
- **Redshift long-term system table retention in S3 Tables (15 Oct):** keeping system table history beyond 7 days, written as Iceberg, is available in more Regions.[43]
- **Kafka 4.4.0 still not released:** on 19 Oct the downloads page still lists 4.3.1 (25 Jun) as the highest version and 4.2.2 (29 Sep) as the newest by date.[44]
- **DuckDB 2.0.0 is still scheduled for 21 Oct (tentative),** with 2.0.1 on 18 Nov; 1.5.6 (28 Sep) is the latest release.[45]

---

1. **CVE Brief, 17 October 2026**, and its records for **CVE-2026-85748**, **CVE-2026-85746** and **CVE-2026-92262** (published 16 Oct 2026, affected and fixed versions, GitHub advisory references).
2. **CVE-2026-97380**, CVE Brief record (published 17 Oct 2026).
3. **CVE-2026-92262**, CVE Brief record (CWE-706).
4. **CVE-2026-85748: routine contexts use-after-free**, TheHackerWire (CVE description and vector), viewed 19 Oct 2026.
5. **CVE-2026-85746**, TheHackerWire (description, vector, CWE-787, MDEV-40329), viewed 19 Oct 2026.
6. **CVE-2026-92262**, TheHackerWire (description, vector, MDEV-40362), viewed 19 Oct 2026.
7. **CVE-2026-97380**, TheHackerWire (description, vector, CWE-620, MDEV-40470), viewed 19 Oct 2026.
8. **CVE-2026-85748**, Strix (vector, CNA version table), viewed 19 Oct 2026.
9. **CVELIST: CVE-2026-85748**, Vulners (CVE record text and version ranges), viewed 19 Oct 2026.
10. **Security Vulnerabilities (CVE) Fixed in MariaDB Enterprise Server**, MariaDB documentation, viewed 19 Oct 2026.
11. **Release Notes for MariaDB Enterprise Server 10.6.28-24**, MariaDB, 7 Sep 2026.
12. **CVE-2026-107815**, Debian security tracker, viewed 19 Oct 2026.
13. **CVE-2026-107817**, Mend.io vulnerability database (published 9 Oct 2026).
14. **Multiple Vulnerabilities in MariaDB Server (AL-2026-144)**, Cyber Security Agency of Singapore, 16 Oct 2026.
15. **MariaDB 10.6.28 release notes**, MariaDB, 13 Aug 2026.
16. **MariaDB Community Server 10.6.28 now available**, Daniel Bartholomew, MariaDB, 13 Aug 2026.
17. **MariaDB Server 12.3, 11.8, 11.4 and 10.11 – Q3 2026 Maintenance Releases, and Goodbye 10.6**, Frédéric Descamps, MariaDB Foundation, 25 Aug 2026.
18. **MariaDB 11.8.9 release notes**, MariaDB, 24 Aug 2026.
19. **MariaDB 12.3.3 changelog**, MariaDB, 24 Aug 2026.
20. **MariaDB 13.0.2 release notes**, MariaDB, 15 Sep 2026.

21. **Debian mariadb changelog, 1:11.8.9+ds-1**, Debian, 29 Aug 2026.
22. **PoC for CVE-2026-92262 / 85746 / 85748 (SeanDishman)**, Sploitus listing of a GitHub repository, 18 Oct 2026.
23. **MDEV-36377**, MariaDB Jira (SYS_REFCURSOR bug, linked to MDEV-20034 "Add support for the pre-defined weak SYS_REFCURSOR").
24. **Identifier to File Name Mapping**, MariaDB documentation.
25. **Identifier to file name mapping**, MariaDB Knowledge Base mirror (section on pre-5.1.6 identifiers and #mysql50#).
26. **ALTER DATABASE**, MariaDB Knowledge Base mirror (UPGRADE DATA DIRECTORY NAME and #mysql50#).
27. **MariaDBに「USAGEだけ」で管理者を乗っ取れる脆弱性**, @IT (ITmedia), 10 Sep 2026.
28. **MariaDB GRANT PROXY privilege escalation**, AI Heartland, 9 Sep 2026.
29. **Amazon RDS for MariaDB now supports community MariaDB minor versions 10.6.28, 10.11.19, 11.4.13, 11.8.9, and 12.3.3**, AWS What's New, 8 Sep 2026.
30. **Under the hood, off the hot path: How D1 scales to millions of databases**, David Laban and Lambros Petrou, Cloudflare blog, 13 Oct 2026.
31. **D1 changelog**, Cloudflare docs (open beta 28 Sep 2023, GA 1 Apr 2024, read replication beta 10 Apr 2025).
32. **Cloudflare announces GA releases for D1, Hyperdrive, and Workers Analytics Engine**, SD Times, 1 Apr 2024.
33. **Zero-latency SQLite storage in every Durable Object**, Kenton Varda, Cloudflare blog, 26 Sep 2024.
34. **D1 limits**, Cloudflare docs, viewed 19 Oct 2026.
35. **Hyperdrive query caching**, Cloudflare docs, viewed 19 Oct 2026.
36. **Introducing Workers KV Instant, powered by Quicksilver**, Cloudflare blog, 1 Oct 2026.
37. **Metastability and Distributed Systems**, Marc Brooker, 24 May 2021 (quoting Bronson et al., HotOS 2021).
38. **Amazon DynamoDB Streams now supports timestamp-based processing**, AWS What's New, 15 Oct 2026.
39. **Apache Iceberg v3 table upgrades and expression defaults (GA)**, Snowflake release notes, 16 Oct 2026.
40. **Inside Hyperdrive: How we solved a rare Hyperdrive MySQL caching bug**, Pranoy Dutta, Cloudflare blog, 14 Oct 2026.
41. **pg_hint_plan v1.9.0 released**, postgresql.org, 18 Oct 2026.
42. **Confluent Cloud release notes**, entry dated 16 Oct 2026.
43. **Amazon Redshift long-term system table retention with Amazon S3 Tables in additional Regions**, AWS What's New, 15 Oct 2026.
44. **Apache Kafka downloads**, viewed 19 Oct 2026.
45. **DuckDB release calendar**, viewed 19 Oct 2026.